```java
import static java.util.Arrays.stream;
import java.util.*;
import java.util.stream.IntStream;

public class SomeTasks {

    public static Map<Character, Integer> findLeastRepetitive1(char[] charList) {
        if (charList == null || charList.length == 0) {
            return null;
        }
        Map<Character, Integer> freqMap = new HashMap<>();
        for (char c : charList) {
            freqMap.put(c, freqMap.getOrDefault(c, 0) + 1);
        }
        int min = Collections.min(freqMap.values());
        Map<Character, Integer> result = new HashMap<>();
        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() == min) {
                result.put(entry.getKey(), entry.getValue());
            }
        }
        return result;
    }


    public static boolean isPalindrome2(String s) {
        Stack <Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            stack.push(c);
        }
        for (char c : s.toCharArray()) {
            if (stack.pop() != c) {
                return false;
            }
        }
        return true;
    }


    public static String longestCommonPrefix(String[] strs) {
        if (strs == null || strs.length == 0) return "";
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix)) {
```

```
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
  }


  public static int pilotReverse(int x) {
    int reversed = 0;
    while (x != 0) {
      int digit = x % 10; // Extract the last digit
      x /= 10; // Remove the last digit from x

      // Check for overflow/underflow before updating reversed
      if (reversed > Integer.MAX_VALUE / 10 || (reversed == Integer.MAX_VALUE / 10 && digit > 7)) {
        return 0; // Overflow
      }
      if (reversed < Integer.MIN_VALUE / 10 || (reversed == Integer.MIN_VALUE / 10 && digit < -8)) {
        return 0; // Underflow
      }

      reversed = reversed * 10 + digit; // Append the digit
    }
    return reversed;
  }


  public static int removeElement(int[] nums, int val) {
    int k = 0;
    for (int i = 0; i < nums.length; i++) {
      if (nums[i] != val) {
        nums[k++] = nums[i];
      }
    }
    return k;
  }


  public static int shortLengthOfLastWord(String s) {
    String[] words = s.trim().split("\\s+");
    return words[words.length - 1].length();
  }
```

```java


    public static int[] plusOne(int[] digits) {
        int size = digits .length;
        for (int i = size - 1;  i >= 0; i--) {
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
            if (i == 0) {
                int[] newDigits = new int[size + 1];
                newDigits[0] = 1;
                for (int j = 0;  j < size; j++) {
                    newDigits[j+1] = digits[j];
                }
                return newDigits;
            }
        }
        return digits;
    }


    public static int mySqrt(int x) {
        if (x < 2)
            return x;
        int left = 1, right = x, ans = 0;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if ((long) mid * mid <= x) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return ans;
    }


    public static void bubbleSort(int[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < arr.length - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
```

```
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) {
            break; // Early termination if no swaps occurred
        }
    }
}


public static Map<String, String[]> sortMapByValueNoStreams(Map<String, String[]> map) {
    class StringArrayComparator implements Comparator<String[]> {
        @Override
        public int compare(String[] o1, String[] o2) {
            if (o1 == null || o2 == null || o1.length == 0 || o2.length == 0) {
                return -10;
            }
            return o1[0].compareTo(o2[0]);
        }
    }

    List<Map.Entry<String, String[]>> entryList = new ArrayList<>(map.entrySet());
    entryList.sort(Map.Entry.comparingByValue(new StringArrayComparator()));

    Map<String, String[]> sortedMap = new LinkedHashMap<>();
    for (Map.Entry<String, String[]> entry : entryList) {
        sortedMap.put(entry.getKey(), entry.getValue());
    }
    return sortedMap;
}


public static void main(String[] args) {
    int[] predefinedSet = { 10, 11, 12, 13, 14, 15, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 };
    IntStream intStream = stream(predefinedSet);
    intStream.filter(e -> e % 2 == 0).distinct().map(e -> e * 2).sorted().forEach(System.out::println);
}
```

```

}

```